

МФТИ

Алгоритмы и структуры данных II

Семинар 13

Деревья (1)

Весна 2026

Строгие подробные решения с доказательствами

Содержание

Обозначения и соглашения	2
Соглашения	2
1 Задача 1. Рёбра, лежащие на каждом пути между двумя вершинами	3
2 Задача 2. Нисходящие пути и диаметр	4
3 Задача 3. Пять оптимизационных задач на дереве	5
4 Задача 4. Разбиение на k равных связных поддеревьев	8
5 Задача 5. Кратчайший маршрут, посещающий все вершины	9
6 Задача 6. LCA с помощью глубин и бинарных подъёмов	10
7 Задача 7. Минимум на пути	11
8 Задача 8. XOR на пути	12
9 Задача 9. Набор векторов с общими префиксами	13
10 Задача 10. Число и взаимное расположение центроидов	14
11 Задача 11. Изоморфизм корневых деревьев без хеширования	15
12 Задача 12. Центры дерева и изоморфизм неориентированных деревьев	16
13 Задача 13. Оптимальное размещение столиц	17
14 Задача 14. Компонента ровно из k вершин после удаления рёбер	19
15 Задача 15. Восстановление неотрицательных весов по соседним расстояниям	20

Обозначения и соглашения

Во всех оценках n обозначает размер входа, если в условии конкретной задачи не сказано иначе. Значение $+\infty$ используется для недостижимого состояния в задачах минимизации, а $-\infty$ — в задачах максимизации. Слово «путь» может означать маршрут с повторениями вершин только там, где это прямо следует из постановки; во всех остальных случаях путь простой.

Соглашения

Если дерево подвешено за корень r , через T_v обозначается поддереву вершины v , через $p(v)$ — её родитель, а через $\text{depth}(v)$ — глубина. Длиной невзвешенного пути называется число его рёбер. Изолированная вершина считается путём длины 0.

1. Задача 1. Рёбра, лежащие на каждом пути между двумя вершинами

Решение. Назовём ребро e *мостом*, если после его удаления число компонент связности графа увеличивается. Сначала одним DFS найдём все мосты. Для древесного ребра $v \rightarrow u$ используем стандартные величины

$$\text{tin}[v], \quad \text{low}[v] = \min \left(\text{tin}[v], \min_{(v,x) - \text{обратное}} \text{tin}[x], \min_{u - \text{сын } v} \text{low}[u] \right).$$

Ребро (v, u) является мостом тогда и только тогда, когда

$$\text{low}[u] > \text{tin}[v]. \quad (1)$$

В мультиграфе нужно хранить номера рёбер и пропускать только то ребро, по которому DFS пришёл в вершину: второе параллельное ребро является обратным и не должно быть потеряно.

Удалим мосты и найдём компоненты связности оставшегося графа. Стянем каждую такую компоненту в одну вершину, а каждый мост оставим ребром между полученными вершинами. Получится *дерево мостов* B . Подвесим его, предподсчитаем глубины и LCA.

Пусть $c(u)$ и $c(v)$ — компоненты исходных вершин. Ответ на запрос равен числу рёбер на единственном пути между ними в B :

$$\text{ans}(u, v) = \text{depth}(c(u)) + \text{depth}(c(v)) - 2 \text{depth}(\text{LCA}(c(u), c(v))). \quad (2)$$

Доказательство корректности. Сначала докажем критерий для одного ребра. Ребро e лежит на каждом u - v -пути тогда и только тогда, когда после удаления e вершины u и v оказываются в разных компонентах. В частности, такое ребро обязательно является мостом. Обратно, если мост разделяет u и v , то любой путь между ними обязан пересечь разрез, а единственным ребром исходного графа в этом разрезе служит данный мост.

Формула (1) верна потому, что $\text{low}[u] \leq \text{tin}[v]$ ровно тогда, когда из поддерева u существует обратное ребро к v или к предку v ; вместе с древесным путём оно даёт обход ребра (v, u) . Если такого ребра нет, удаление (v, u) отделяет всё DFS-поддерево u .

После стягивания компонент без мостов граф B связан. Цикла в нём быть не может: любое ребро цикла можно обойти по остальной части цикла, а значит, соответствующее исходное ребро не было бы мостом. Поэтому B — дерево. Его путь от $c(u)$ до $c(v)$ состоит в точности из мостов, разделяющих u и v . По первому абзацу это и есть все рёбра, лежащие на каждом u - v -пути. Формула (2) считает их число. $\square$

Сложность. Поиск мостов, компонент и построение B занимают $\mathcal{O}(n + m)$. Бинарные подъёмы для LCA требуют $\mathcal{O}(n \log n)$ времени и памяти; один запрос обрабатывается за $\mathcal{O}(\log n)$. С RMQ для LCA время запроса можно уменьшить до $\mathcal{O}(1)$.

2. Задача 2. Нисходящие пути и диаметр

Решение. Обрабатываем вершины в обратном порядке DFS. Для листа полагаем $dp[v] = 0$. Для любой другой вершины

$$dp[v] = \max \left(0, \max_{u: p(u)=v} \{1 + dp[u]\} \right). \quad (3)$$

Таким образом, $dp[v]$ — наибольшее число рёбер в пути, который начинается в v и всё время идёт к потомкам.

Для каждой вершины выпишем значения $1 + dp[u]$ для всех её детей и дополним список двумя нулями. Пусть два наибольших значения равны $a_v \geq b_v$. Тогда длина диаметра равна

$$D = \max_v (a_v + b_v). \quad (4)$$

Для восстановления самого пути достаточно вместе с двумя максимумами хранить детей, в которых они достигаются, а затем пройти по сохранённым указателям до обоих концов.

Доказательство корректности. Индукцией по высоте поддерева докажем (3). У листа единственный нисходящий путь состоит из самого листа и имеет нулевую длину. Из внутренней вершины путь либо никуда не идёт, либо первым ребром переходит в некоторого ребёнка u , после чего оптимально продолжить путь длины $dp[u]$. Максимум ровно и даёт (3).

Рассмотрим произвольный простой путь P и его вершину v наименьшей глубины. Части P , выходящие из v , идут не более чем в два разных дочерних поддеревья; если конец пути равен v , одна часть имеет длину 0. Поэтому

$$|P| \leq a_v + b_v.$$

Следовательно, правая часть (4) не меньше длины любого пути. Обратно, два нисходящих пути, задающие a_v и b_v , лежат в разных дочерних поддеревьях и пересекаются только в v ; их объединение является простым путём длины $a_v + b_v$. Значит, максимум в (4) действительно равен диаметру. $\square$

Сложность. Каждое ребро просматривается постоянное число раз: $\mathcal{O}(n)$ времени и $\mathcal{O}(n)$ памяти.

3. Задача 3. Пять оптимизационных задач на дереве

Пункт а: максимальное паросочетание

Решение. Для вершины v введём два значения:

- $M_0[v]$ — максимум в T_v , если ребро к родителю не выбрано;
- $M_1[v]$ — максимум в T_v , если v уже соединена ребром паросочетания с родителем.

Ребро к родителю в число $M_1[v]$ не включаем. Если v уже занята, ни одно ребро к ребёнку выбрать нельзя, поэтому

$$M_1[v] = \sum_{u - \text{ребёнок } v} M_0[u]. \quad (5)$$

Если v свободна, можно либо не соединять её ни с одним ребёнком, либо соединить ровно с одним ребёнком u :

$$M_0[v] = \max \left\{ \sum_z M_0[z], \max_u \left(1 + M_1[u] + \sum_{z \neq u} M_0[z] \right) \right\}. \quad (6)$$

Ответ равен $M_0[r]$.

Доказательство корректности. В паросочетании степень каждой вершины не превосходит 1. Поэтому в состоянии $M_1[v]$ все дочерние рёбра запрещены, что доказывает (5). В состоянии $M_0[v]$ допустимы ровно варианты из (6): ноль либо одно выбранное дочернее ребро. После фиксации этого выбора дочерние поддеревья не имеют общих вершин, и их оптимумы складываются. Перебраны все и только допустимые варианты, поэтому индукция по поддереву доказывает обе формулы. $\square$

Пункт б: минимальное вершинное покрытие

Решение. Пусть $C_0[v]$ и $C_1[v]$ — минимальные размеры покрытия T_v при условии, что v , соответственно, не взята и взята. Тогда

$$\begin{aligned} C_1[v] &= 1 + \sum_u \min(C_0[u], C_1[u]), \\ C_0[v] &= \sum_u C_1[u]. \end{aligned} \quad (7)$$

Ответ: $\min(C_0[r], C_1[r])$.

Доказательство корректности. Если v взята, каждое ребро vu уже покрыто со стороны v , и в поддереве u разрешено лучшее из двух состояний. Если v не взята, для покрытия каждого ребра vu ребёнок u обязан быть взят. Иных ограничений между разными поддеревьями нет, поэтому суммы в (7) точны. $\square$

Пункт в: максимальное независимое множество

Решение. Аналогично обозначим через $I_0[v]$ и $I_1[v]$ наибольший размер независимого множества в T_v , когда v не взята и взята:

$$\begin{aligned} I_1[v] &= 1 + \sum_u I_0[u], \\ I_0[v] &= \sum_u \max(I_0[u], I_1[u]). \end{aligned} \quad (8)$$

Ответ: $\max(I_0[r], I_1[r])$.

Доказательство корректности. Если v принадлежит независимому множеству, ни один соседний ребёнок принадлежать ему не может. Если v не взята, состояние каждого ребёнка можно выбрать независимо. Это в точности два перехода (8). $\square$

Пункт г: максимальная клика

Решение. Если $n = 1$, ответ равен 1. Если $n \geq 2$, ответ равен 2, а любое ребро задаёт максимальную клику.

Доказательство корректности. В дереве нет циклов, в частности, нет треугольников. Поэтому три попарно смежные вершины существовать не могут и размер клики не превосходит 2. При $n \geq 2$ в связном дереве есть ребро, то есть клика размера 2 существует. $\square$

Пункт д: покрытие минимальным числом непересекающихся путей

Решение. Выберем некоторое множество рёбер F . Так как исходный граф — дерево, граф (V, F) не содержит циклов. Его компоненты являются путями тогда и только тогда, когда степень каждой вершины в F не превосходит 2. Если условие выполнено, число компонент-путей равно

$$n - |F|. \quad (9)$$

Значит, нужно максимизировать число выбранных рёбер при ограничении на степени.

Пусть $P[v][p]$, где $p \in \{0, 1\}$, — максимум числа выбранных рёбер, целиком лежащих в T_v , если ребро $p(v)v$ выбрано ровно при $p = 1$. Само родительское ребро в значение не входит. Начнём с

$$S = \sum_u P[u][0].$$

Если выбрать ребро vu , прибавка вместо невыбранного варианта равна

$$\Delta_u = 1 + P[u][1] - P[u][0]. \quad (10)$$

У вершины v можно выбрать не более $2 - p$ дочерних рёбер. Поэтому к S добавляем не более $2 - p$ наибольших положительных величин Δ_u . Ответ:

$$n - P[r][0]. \quad (11)$$

Доказательство корректности. Компонента ациклического графа максимальной степени не более 2 есть либо изолированная вершина, либо простой путь. Для леса число компонент равно числу вершин минус число рёбер, откуда следует (9).

При фиксированном состоянии p у вершины остаётся $2 - p$ свободных места для дочерних рёбер. Если ребро vu не выбрано, оптимальный вклад поддерева равен $P[u][0]$; если выбрано — $1 + P[u][1]$. Разность равна (10), поэтому выгодно взять ровно наибольшие положительные прибавки, но не больше разрешённого числа. Дочерние поддеревья не пересекаются, так что иных зависимостей нет. Индукция доказывает корректность динамики, а (9) превращает найденный максимум в ответ (11). $\square$

Сложность. Все пять алгоритмов работают за $\mathcal{O}(n)$ времени и $\mathcal{O}(n)$ памяти. В пункте д) в каждой вершине достаточно поддерживать две наибольшие положительные прибавки, не сортируя весь список детей.

4. Задача 4. Разбиение на k равных связных поддеревьев

Решение. Если n не делится на k , ответ отрицательный. Иначе требуемый размер каждой части равен

$$s = \frac{n}{k}.$$

Подвесим дерево за произвольный корень и выполним DFS снизу вверх. Вершина v получает от каждого ребёнка число неприкреплённых вершин его поддерева. Положим

$$q_v = 1 + \sum_{u - \text{ребёнок } v} q_u. \quad (12)$$

Здесь ребёнок возвращает 0, если его остаток уже образовал готовую компоненту размера s .

- Если $q_v < s$, возвращаем q_v родителю.
- Если $q_v = s$, объявляем эти вершины очередной частью, при $v \neq r$ разрезаем ребро $p(v)v$ и возвращаем 0.
- Если $q_v > s$, разбиение невозможно.

После обработки корня должно получиться $q_r = s$, то есть после закрытия корня он также возвращает 0. Записанные разрезанные рёбра задают искомое разбиение.

Доказательство корректности. Докажем индукцией следующий инвариант. После обработки T_v все уже закрытые части имеют размер s , а незакрытые вершины, если они есть, образуют единственное связное множество размера $q_v < s$, содержащее v . Только оно может быть соединено с вершинами вне T_v , потому что единственное выходящее из T_v ребро инцидентно v .

Для листа утверждение очевидно. Для внутренней вершины все незакрытые остатки детей обязаны присоединиться к v : если отрезать остаток ребёнка, он останется компонентой размера меньше s . Их объединение вместе с v связно и имеет размер (12). При размере s его можно и нужно закрыть; при меньшем размере его необходимо передать выше; при размере больше s компоненту допустимого размера уже получить нельзя. Тем самым переход и необходим, и достаточен.

У корня нет родителя, поэтому допустимое разбиение существует ровно тогда, когда последний остаток также закрылся. Суммарно закрыто $n/s = k$ компонент. $\square$

Сложность. $\mathcal{O}(n)$ времени и $\mathcal{O}(n)$ памяти на дерево и стек DFS.

5. Задача 5. Кратчайший маршрут, посещающий все вершины

Решение. Пусть W — сумма весов всех рёбер. Сначала зафиксируем начальную вершину a и конечную вершину b . Оптимальная длина маршрута, посещающего все вершины, равна

$$L(a, b) = 2W - \text{dist}(a, b). \quad (13)$$

Действительно, рёбра на пути $a \rightsquigarrow b$ достаточно пройти по одному разу, а все остальные — по два раза.

Из (13) следуют все естественные варианты постановки.

- Если начало и конец свободны, выбираем концы диаметра:

$$L_{\min} = 2W - D.$$

- Если начало r задано, а конец свободен (именно этот вариант используется в задаче Codeforces 61D), выбираем наиболее далёкую от r вершину:

$$L_{\min} = 2W - \max_v \text{dist}(r, v).$$

- Если требуется вернуться в начальную вершину, ответ равен $2W$.

Сам маршрут строится так. Идём вдоль выбранного пути $a \rightsquigarrow b$. В каждой его вершине полностью обходим каждое боковое поддерево и возвращаемся, после чего переходим к следующей вершине пути и назад по этому ребру уже не идём.

Доказательство корректности. Удаление любого ребра e , не лежащего на пути $a \rightsquigarrow b$, оставляет a и b в одной компоненте. Чтобы посетить вершины другой компоненты и закончить в b , маршрут обязан пересечь e туда и обратно, то есть не менее двух раз. Если e лежит на $a \rightsquigarrow b$, начало и конец находятся по разные стороны от него, поэтому пересечь e нужно нечётное и, в частности, не меньшее одного число раз. Получаем нижнюю границу

$$2 \sum_{e \notin a \rightsquigarrow b} w_e + \sum_{e \in a \rightsquigarrow b} w_e = 2W - \text{dist}(a, b).$$

Описанный обход проходит каждое боковое ребро ровно дважды, а каждое ребро пути $a \rightsquigarrow b$ ровно один раз, поэтому достигает нижней границы. Формула (13) доказана. При свободном выборе концов её нужно минимизировать, то есть максимизировать расстояние между концами; это по определению диаметр. При фиксированном начале достаточно выбрать самую далёкую вершину. $\square$

Сложность. Сумма весов, наиболее далёкая вершина и диаметр дерева находятся за $O(n)$; построение маршрута также занимает линейное время плюс время на вывод самого маршрута.

6. Задача 6. LCA с помощью глубин и бинарных подъёмов

Решение. Положим $K = \lfloor \log_2 n \rfloor$. Во время DFS вычислим глубины и таблицу

$$up[v][0] = p(v), \quad up[v][j] = up[up[v][j-1]][j-1]. \quad (14)$$

Корень считаем собственным предком: $p(r) = r$.

Для запроса $LCA(u, v)$:

1. если $\text{depth}(u) < \text{depth}(v)$, меняем вершины местами;
2. поднимаем u на разность глубин, раскладывая её по степеням двойки;
3. если теперь $u = v$, возвращаем эту вершину;
4. перебираем $j = K, K-1, \dots, 0$. Если $up[u][j] \neq up[v][j]$, одновременно заменяем u и v на эти 2^j -е предки;
5. возвращаем $p(u)$.

Доказательство корректности. Из (14) индукцией по j следует, что $up[v][j]$ есть предок v на расстоянии 2^j . Поэтому второй шаг точно выравнивает глубины.

После выравнивания, если вершины равны, это и есть их самый глубокий общий предок. Иначе настоящий LCA находится строго выше обеих. На четвёртом шаге мы совершаем подъём только тогда, когда полученные предки различны; следовательно, ни одна вершина не поднимается до LCA или выше него. Перебор степеней от больших к меньшим оставляет в конце две различные вершины, родители которых уже совпадают: если бы можно было поднять обе хотя бы на одно ребро и сохранить различие, сработал бы случай $j = 0$. Общий родитель и является LCA. $\square$

Сложность. Предподсчёт занимает $\mathcal{O}(n \log n)$ времени и памяти, один запрос — $\mathcal{O}(\log n)$.

7. Задача 7. Минимум на пути

Решение. Вместе с таблицей up построим

$$mn[v][j] = \min\{\text{веса рёбер на пути от } v \text{ до } up[v][j]\}.$$

База:

$$mn[v][0] = w(v, p(v)),$$

а для корня берём $+\infty$. Переход:

$$mn[v][j] = \min(mn[v][j-1], mn[up[v][j-1]][j-1]). \quad (15)$$

В запросе для u, v выполняем те же подъёмы, что и при поиске LCA. Каждый раз, поднимая вершину x на 2^j , добавляем к ответу $mn[x][j]$ операцией минимума. Сначала поднимаем более глубокую вершину. Если вершины не совпали, поднимаем их одновременно от больших степеней к меньшим, когда их 2^j -е предки различны. В конце отдельно учитываем рёбра от обеих вершин к общему родителю. Для запроса $u = v$ путь не содержит рёбер; можно вернуть $+\infty$ либо специально оговорённое пустое значение.

Доказательство корректности. Путь длины 2^j из v к предку распадается на две последовательные половины длины 2^{j-1} , поэтому (15) хранит минимум ровно на нужном отрезке. Алгоритм LCA разлагает путь $u \rightsquigarrow \text{LCA}(u, v)$ и путь $v \rightsquigarrow \text{LCA}(u, v)$ на непересекающиеся блоки степенных длин. Мы берём минимум на каждом блоке, а минимум минимумов равен минимуму на объединении, то есть на всём пути $u \rightsquigarrow v$. $\square$

Сложность. $\mathcal{O}(n \log n)$ времени и памяти на предподсчёт, $\mathcal{O}(\log n)$ на запрос.

8. Задача 8. XOR на пути

Решение. Выберем корень r и одним DFS вычислим

$$px[v] = \bigoplus_{e \in r \rightsquigarrow v} w_e, \quad px[r] = 0. \quad (16)$$

Для любых u, v ответ равен

$$\boxed{px[u] \oplus px[v]}. \quad (17)$$

Доказательство корректности. Пути $r \rightsquigarrow u$ и $r \rightsquigarrow v$ имеют общий префикс $r \rightsquigarrow \text{LCA}(u, v)$. Вес каждого ребра этого префикса входит в (17) дважды и уничтожается по тождеству $x \oplus x = 0$. Все остальные рёбра входят ровно один раз; они и образуют путь $u \rightsquigarrow v$. Поэтому (17) даёт требуемый XOR. $\square$

Сложность. $\mathcal{O}(n)$ времени и памяти на предподсчёт, $\mathcal{O}(1)$ на запрос.

9. Задача 9. Набор векторов с общими префиксами

Решение. Каждый вектор будем представлять парой

$$(\text{tail}, \text{len}).$$

Элемент, добавленный операцией `push_back`, хранится в неизменяемой вершине x . В ней записаны значение, указатель $\text{jump}[x][0]$ на предыдущий элемент и бинарные подёмы

$$\text{jump}[x][j] = \text{jump}[\text{jump}[x][j-1]][j-1]. \quad (18)$$

Все векторы могут ссылаться на одни и те же неизменяемые вершины.

Определим $\text{climb}(x, t)$ как подъём на t предыдущих элементов по двоичному разложению t .

- i -й элемент j -го вектора при нумерации с единицы хранится в $\text{climb}(\text{tail}_j, \text{len}_j - i)$.
- `pop_back`: заменяем хвост на его непосредственного предка и уменьшаем длину на 1.
- `push_back(x)`: создаём одну новую вершину, строим для неё строку (18), назначаем её хвостом и увеличиваем длину.
- Новый префикс длины k : создаём только новый описатель

$$(\text{climb}(\text{tail}_j, \text{len}_j - k), k).$$

При $k = 0$ хвост равен специальной нулевой вершине.

Изменение описателя одного вектора не меняет ни одной общей вершины и потому не затрагивает остальные векторы.

Доказательство корректности. Поддерживаем инвариант: если идти от `tail` по указателям на предыдущий элемент, получаются элементы вектора в обратном порядке. Он верен для пустого вектора. `push_back` ставит новую вершину перед старой обратной цепочкой, а `pop_back` удаляет её голову, поэтому инвариант сохраняется. Подъём на $\text{len} - i$ шагов оставляет ровно i -й элемент. А подъём на $\text{len} - k$ шагов отбрасывает все элементы после первых k , следовательно, новый описатель задаёт требуемый префикс. Формула (18) корректно выполняет любой такой подъём по двоичному разложению числа шагов. $\square$

Сложность. После q запросов создано не более q узлов и требуется $\lfloor \log_2 q \rfloor + 1$ уровней. Каждая операция занимает $\mathcal{O}(\log q)$ времени, суммарно $\mathcal{O}(q \log q)$; память $\mathcal{O}(q \log q)$.

10. Задача 10. Число и взаимное расположение центроидов

Решение. Напомним: вершина c является центроидом, если после её удаления размер каждой компоненты не превосходит $n/2$.

Пусть $u \neq v$ — два центроида. Подвесим дерево за u , и пусть w — ребёнок u , лежащий на пути к v . Обозначим размеры поддеревьев через sz . После удаления u компонентой, содержащей v , является T_w , поэтому

$$sz[w] \leq \frac{n}{2}. \quad (19)$$

После удаления v компонента, содержащая u , имеет размер $n - sz[v]$. Так как v — центроид,

$$n - sz[v] \leq \frac{n}{2}, \quad \text{то есть} \quad sz[v] \geq \frac{n}{2}. \quad (20)$$

Но $T_v \subseteq T_w$, откуда

$$sz[v] \leq sz[w]. \quad (21)$$

Из (19)–(21) следует $sz[v] = sz[w] = n/2$. Если бы $v \neq w$, включение $T_v \subset T_w$ было бы строгим и давало бы $sz[v] < sz[w]$, что невозможно. Значит, $v = w$, то есть два различных центроида соединены ребром.

Доказательство корректности. Все использованные неравенства непосредственно следуют из определения центроида и устройства компоненты после удаления вершины; вместе они доказывают смежность любых двух различных центроидов. Если бы центроидов было хотя бы три, каждая пара из них должна была бы быть соединена ребром, то есть дерево содержало бы треугольник. В дереве циклов нет, поэтому центроидов не больше двух. $\square$

Сложность. Сам центроид находится за $\mathcal{O}(n)$: вычисляем размеры поддеревьев и проверяем для каждой вершины максимальный размер компоненты после её удаления.

11. Задача 11. Изоморфизм корневых деревьев без хеширования

Решение. Тип листа обозначим одним целым идентификатором. После того как типы всех детей вершины v известны, её *сигнатура* есть отсортированный список

$$\sigma(v) = \text{sort}(\text{id}(u) : u \text{ — ребёнок } v). \quad (22)$$

Равным спискам назначаем один идентификатор, различным — разные. Обращать нужно оба сравниваемых дерева совместно, иначе номера типов могут оказаться несогласованными.

Осталось выполнить точное, а не хеш-вероятностное сжатие списков. Вершины обрабатываем снизу вверх. Все списки (22) вставляем в обычный бор над алфавитом целых идентификаторов; переходы из вершины бора храним в сбалансированном дереве поиска. Одинаковые последовательности приходят в одну и ту же терминальную вершину бора и получают один тип. Разные последовательности расходятся на первой отличающейся позиции или имеют разную длину и потому получают разные типы.

Корневые деревья изоморфны тогда и только тогда, когда идентификаторы их корней совпали. Само соответствие можно восстановить, сопоставляя у равнотипных вершин детей одинаковых типов.

Доказательство корректности. Индукцией по высоте докажем, что две вершины получают одинаковый тип ровно тогда, когда изоморфны корневые поддеревья с этими корнями. Для листьев это очевидно. Для внутренних вершин корневой изоморфизм обязан задавать биекцию между детьми и сопоставлять изоморфные дочерние поддеревья. По предположению индукции это равносильно совпадению мультимножеств идентификаторов детей, то есть совпадению отсортированных списков (22). Бор сравнивает списки точно, поэтому назначает один тип ровно в этом случае. Утверждение доказано, в частности, для корней. $\square$

Сложность. Суммарная длина всех дочерних списков равна числу рёбер, то есть $\mathcal{O}(n)$. Их сортировка занимает

$$\sum_v \mathcal{O}(\deg(v) \log \deg(v)) = \mathcal{O}(n \log n).$$

Каждый символ списка создаёт или проходит один переход бора стоимостью $\mathcal{O}(\log n)$. Итого $\mathcal{O}(n \log n)$ времени и $\mathcal{O}(n)$ узлов памяти. Случайных совпадений, свойственных хешированию, нет.

12. Задача 12. Центры дерева и изоморфизм неориентированных деревьев

Решение. Возьмём диаметр $a \rightsquigarrow b$ длины D . Его середина — одна вершина при чётном D и две соседние вершины при нечётном D .

Покажем, что это единственно возможный набор центров. Для любой вершины x

$$D = \text{dist}(a, b) \leq \text{dist}(a, x) + \text{dist}(x, b),$$

поэтому

$$\max(\text{dist}(x, a), \text{dist}(x, b)) \geq \left\lceil \frac{D}{2} \right\rceil. \quad (23)$$

Пусть c — середина диаметра (при нечётном D берём любую из двух), а y — произвольная вершина. Обозначим через z место присоединения пути от y к диаметру. Если $t = \text{dist}(a, z)$ и $h = \text{dist}(y, z)$, то пути $y \rightsquigarrow a$ и $y \rightsquigarrow b$ не длиннее диаметра:

$$h + t \leq D, \quad h + D - t \leq D.$$

Следовательно, $h \leq \min(t, D - t)$, откуда

$$\text{dist}(c, y) = h + \text{dist}(c, z) \leq \left\lceil \frac{D}{2} \right\rceil. \quad (24)$$

Итак, середина диаметра имеет минимально возможный эксцентриситет $\lceil D/2 \rceil$. Равенство в нижней границе (23) возможно только в середине пути $a \rightsquigarrow b$: уход с этого пути добавляет положительное расстояние сразу к обоим концам. Значит, множество вершин минимального эксцентриситета — ровно одна середина или две соседние середины фиксированного диаметра.

Середина любого другого диаметра по тому же доказательству имеет минимальный эксцентриситет, поэтому принадлежит этому же набору. Тем самым центр в смысле условия либо единственный, либо центров два и они смежны.

Для проверки изоморфизма неориентированных деревьев:

1. находим центр каждого дерева, например, по диаметру;
2. если числа центров различны, деревья не изоморфны;
3. при одном центре подвешиваем оба дерева за центр и применяем алгоритм задачи 11;
4. при двух центрах удаляем центральное ребро, вставляем в его середину новую искусственную вершину и соединяем её с обоими концами. Полученные деревья сравниваем как корневые с искусственным корнем.

Доказательство корректности. Формулы (23)–(24) доказывают утверждение о числе и расположении центров. Любой изоморфизм сохраняет расстояния, диаметры и их середины, поэтому он обязан переводить центр в центр и центральное ребро в центральное ребро. После канонического выбора одного центра или после подразделения центрального ребра неориентированный изоморфизм в точности превращается в корневой изоморфизм. Корректность последнего доказана в задаче 11. $\square$

Сложность. Центры находятся за $\mathcal{O}(n)$, сравнение — за $\mathcal{O}(n \log n)$ времени и $\mathcal{O}(n)$ памяти.

13. Задача 13. Оптимальное размещение столиц

Решение. Запишем K вместо цены объявления столицы, чтобы не путать её с индексами. Для столицы стоимость обслуживания самой себя равна 0; для другой вершины x , обслуживаемой столицей g , она равна

$$c(x, g) = d_{\text{dist}(x, g)}. \quad (25)$$

Переход к связным областям. Для фиксированного множества столиц каждую вершину назначим ближайшей столице, а равенства разрешим одним фиксированным порядком столиц. Область каждой столицы связна. Действительно, если x назначена столице g , то для любой вершины y на пути $x \rightsquigarrow g$ и любой другой столицы h

$$\text{dist}(y, h) \geq \text{dist}(x, h) - \text{dist}(x, y) \geq \text{dist}(x, g) - \text{dist}(x, y) = \text{dist}(y, g).$$

При равенстве тот же порядок выбирает g . Значит, весь путь от x к g лежит в её области.

Поэтому оптимум можно искать среди разбиений дерева удалением рёбер на связные компоненты, в каждой из которых выбрана ровно одна столица. Обратно, если произвольное такое разбиение назначает вершину не ближайшей из выбранных столиц, переназначение к ближайшей не увеличивает цену, поскольку d_i не убывает. Следовательно, оптимумы двух формулировок совпадают.

Динамика. Подвесим дерево за произвольный корень r . Предпосчитаем все расстояния между вершинами и времена входа/выхода DFS, чтобы за $\mathcal{O}(1)$ проверять $g \in T_u$.

Состояние $F[v, g]$ имеет следующий точный смысл. Внутри T_v некоторые рёбра разрезаны. Все компоненты, не содержащие v , уже закрыты, имеют свою столицу и полностью оплачены. Компонента, содержащая v , пока открыта: её единственная столица должна быть g , а цена K за эту столицу ещё не прибавлена. Если $g \notin T_v$, путь к ней выходит через ребро $vp(v)$. Стоимости обслуживания всех вершин T_v , включая открытую компоненту, уже учтены.

Пусть

$$A[v] = K + \min_{g \in T_v} F[v, g] \quad (26)$$

— цена полностью закрытого поддерева T_v , когда его верхняя компонента получает столицу внутри T_v .

Для листа $F[v, g] = c(v, g)$. В общем случае начинаем с $c(v, g)$ и независимо рассматриваем детей u .

- Если $g \in T_u$, ребро vu обязано остаться: иначе v была бы отсоединена от столицы g . Вклад равен $F[u, g]$.
- Если $g \notin T_u$, ребро можно оставить, получив вклад $F[u, g]$, либо разрезать и полностью решить T_u за $A[u]$.

Итак,

$$F[v, g] = c(v, g) + \sum_{u \text{ — ребёнок } v} \begin{cases} F[u, g], & g \in T_u, \\ \min(F[u, g], A[u]), & g \notin T_u. \end{cases} \quad (27)$$

Вычисляем вершины снизу вверх; после строки $F[v, *]$ вычисляем (26). Ответ равен

$$\boxed{A[r]}. \quad (28)$$

Для восстановления решения храним выборы в минимумах. Начинаем с минимизирующей (26) столицы корня. При выборе $A[u]$ объявляем новой столицей соответствующий минимуму $h \in T_u$; при выборе $F[u, g]$ продолжаем текущую область со столицей g .

Доказательство корректности. Связность областей ближайших столиц и обратное переназначение, доказанные выше, показывают, что достаточно и необходимо оптимизировать по связным компонентам с одной столицей.

Теперь индукцией по размеру T_v докажем смысл состояния $F[v, g]$. У листа нет внутренних рёбер, и единственная вершина открытой компоненты платит ровно $c(v, g)$. Пусть утверждение верно для детей v . Если столица g лежит в T_u , единственный путь от v к g проходит через vu , поэтому разрезать это ребро нельзя. Если $g \notin T_u$, возможны ровно два случая: ребро сохранено и часть компоненты u обслуживается той же столицей g , либо ребро удалено и всё T_u становится независимым закрытым решением. По предположению индукции их точные оптимальные цены равны $F[u, g]$ и $A[u]$. Поддеревья детей попарно не пересекаются, поэтому после фиксации сохранения или удаления рёбер их цены складываются. Это доказывает (27).

При закрытии верхней компоненты её столица обязана лежать в T_v , а неучтённая до сих пор цена её открытия равна K ; отсюда (26). У корня нет родительского продолжения, поэтому его открытая компонента обязана закрыться. Формула (28) перебирает все допустимые разбиения и выбирает лучшее, а значит, по первому абзацу даёт оптимальное размещение столиц. $\square$

Сложность. Все расстояния в дереве можно найти n запусками DFS за $\mathcal{O}(n^2)$. Состояний $F[v, g]$ ровно n^2 ; суммарно просмотрено

$$\sum_v n \cdot \#\{\text{дети } v\} = n(n-1) = \mathcal{O}(n^2)$$

переходов. Вычисление всех минимумов (26) также занимает $\mathcal{O}(n^2)$. Итого $\mathcal{O}(n^2)$ времени и памяти, что сильнее требуемой оценки $\mathcal{O}(n^3)$.

14. Задача 14. Компонента ровно из k вершин после удаления рёбер

Решение. Подвесим дерево за произвольный корень r . Для каждой вершины v построим массив

$$dp_v[s] = \begin{array}{l} \text{минимальное число удалённых рёбер внутри } T_v, \text{ при котором} \\ \text{компонента, содержащая } v, \text{ имеет ровно } s \text{ вершин.} \end{array} \quad (29)$$

Другие компоненты внутри T_v могут иметь любые размеры. Изначально, до обработки детей,

$$dp_v[1] = 0, \quad dp_v[s] = +\infty \quad (s \neq 1).$$

Присоединяем детей по одному обычным рюкзачным слиянием. Пусть уже обработана часть детей и текущая компонента имеет размер a . Для очередного ребёнка u есть два типа перехода:

$$ndp[a] = \min(ndp[a], dp[a] + 1), \quad \text{удалить ребро } vu; \quad (30)$$

$$ndp[a + b] = \min(ndp[a + b], dp[a] + dp_u[b]), \quad \text{оставить } vu, 1 \leq b \leq k - a. \quad (31)$$

Размеры больше k хранить не требуется.

После вычисления всех таблиц компоненту размера k можно выбрать с верхней вершиной v . Внутри T_v нужно $dp_v[k]$ разрезов, а при $v \neq r$ дополнительно удалить ребро к родителю. Поэтому

$$\text{ans} = \min_v (dp_v[k] + [v \neq r]). \quad (32)$$

Доказательство корректности. Докажем (29) индукцией по числу обработанных детей. До их обработки компонента содержит только v , что задаёт базу. Для ребёнка u ребро vu либо удалено, либо сохранено. В первом случае ни одна вершина T_u не входит в компоненту v , и платится ровно один новый разрез (30). Во втором случае в компоненту входит связная компонента, содержащая u , некоторого размера b ; её минимальная внутренняя цена по предположению индукции равна $dp_u[b]$, что даёт (31). Третьего случая нет. Разные дочерние поддеревья не пересекаются, поэтому рюкзачное сложение цен точно.

У любой искомой компоненты есть единственная вершина v минимальной глубины. Вся компонента лежит в T_v , а если $v \neq r$, ребро к её родителю обязательно удалено. Поэтому её цена не меньше соответствующего слагаемого (32). Обратно, состояние $dp_v[k]$ вместе с удалением родительского ребра действительно изолирует связную компоненту ровно из k вершин. Минимум (32) точен. $\square$

Сложность. Прямые слияния требуют $\mathcal{O}(nk^2) \subseteq \mathcal{O}(n^3)$ времени и $\mathcal{O}(nk) \subseteq \mathcal{O}(n^2)$ памяти.

15. Задача 15. Восстановление неотрицательных весов по соседним расстояниям

Решение. Подвесим данное дерево за вершину 1. Пусть

$$x_v = \text{dist}(1, v), \quad x_1 = 0,$$

а

$$\ell_i = \text{LCA}(i, i+1).$$

Из формулы расстояния в корневом дереве получаем систему

$$x_i + x_{i+1} - 2x_{\ell_i} = d_i,$$

или, что будет удобнее,

$$x_{i+1} = d_i - x_i + 2x_{\ell_i}. \quad (33)$$

Все ℓ_i находим бинарными подъёмами за $\mathcal{O}(n \log n)$.

Подъём по степеням двойки. Будем последовательно восстанавливать все x_v по модулям $2, 4, 8, \dots$. Перед переходом к модулю $q = 2^b$ известен массив $\text{old}[v] \equiv x_v \pmod{q/2}$. Для $q = 2$ начинаем с нулевого массива по модулю 1. Новый массив cur вычисляем в порядке номеров вершин:

$$\begin{aligned} \text{cur}[1] &= 0, \\ \text{cur}[i+1] &= (d_i - \text{cur}[i] + 2\text{old}[\ell_i]) \pmod{q}, \quad i = 1, \dots, n-1. \end{aligned} \quad (34)$$

Остаток всегда нормализуем в диапазон $[0, q)$, после чего заменяем $\text{old} \leftarrow \text{cur}$.

Пусть $C = \max_i d_i$. Продолжаем, пока модуль M не станет строго больше nC (при $C = 0$ достаточно взять любой $M > 0$, например 2). Полученные остатки считаем кандидатами на точные значения x_v . Для каждого некорневого v восстанавливаем вес родительского ребра:

$$w(p(v), v) = x_v - x_{p(v)}. \quad (35)$$

Если хотя бы один вес отрицателен, ответа нет. Наконец, обязательно пересчитываем все данные расстояния по формуле

$$x_i + x_{i+1} - 2x_{\ell_i} \quad (36)$$

и сравниваем их с d_i . При несовпадении выводим, что решения нет; иначе веса (35) являются ответом. Если бы в исходной версии требовались положительные веса, в проверке (35) следовало бы заменить $w \geq 0$ на $w > 0$; в данном листке разрешён ноль.

Доказательство корректности. Сначала докажем корректность одного шага (34). Предположим, что существует решение x и

$$\text{old}[v] \equiv x_v \pmod{q/2}$$

для всех v . Тогда

$$2\text{old}[v] \equiv 2x_v \pmod{q}. \quad (37)$$

Индукцией по i докажем $cur[i] \equiv x_i \pmod{q}$. Для $i = 1$ это следует из $x_1 = 0$. Если утверждение верно для i , то (33), (34) и (37) дают

$$cur[i + 1] \equiv d_i - x_i + 2x_{\ell_i} = x_{i+1} \pmod{q}.$$

Следовательно, после каждого шага получены правильные остатки любого существующего решения.

Теперь обоснуем, почему достаточно модуля $M > nC$. Рассмотрим любое ребро e . После его удаления множество меток $1, 2, \dots, n$ разбивается на два непустых множества. При последовательном просмотре меток от 1 до n сторона разреза хотя бы один раз меняется; значит, существуют соседние номера $i, i + 1$, лежащие по разные стороны. Их путь содержит e , поэтому из неотрицательности весов

$$0 \leq w_e \leq d_i \leq C. \quad (38)$$

Любой путь от корня содержит не более $n - 1$ рёбер, а значит,

$$0 \leq x_v \leq (n - 1)C < nC < M. \quad (39)$$

Правильный остаток $x_v \bmod M$ в диапазоне $[0, M)$ по (39) равен самому x_v . Итак, если решение существует, алгоритм восстанавливает его точные корневые расстояния, веса (35) неотрицательны, а проверка (36) успешна.

Если алгоритм принял кандидата, (35) задаёт неотрицательные веса. Телескопическая сумма этих разностей на пути от корня до v равна x_v ; поэтому (36) есть настоящее расстояние между i и $i + 1$. Финальная проверка гарантирует совпадение со всеми входными данными, то есть выведенный ответ допустим. Если проверка не прошла, существующего решения быть не могло, иначе предыдущий абзац гарантировал бы его точное восстановление и успешную проверку. $\square$

Сложность. LCA и значения ℓ_i требуют $\mathcal{O}(n \log n)$ времени. Число модулей равно

$$\mathcal{O}(\log(nC + 1)) = \mathcal{O}(\log n + \log(C + 1)),$$

каждый проход (34) линеен. Итого

$$\mathcal{O}(n \log n + n \log(C + 1))$$

времени. Память — $\mathcal{O}(n \log n)$ для таблицы LCA и $\mathcal{O}(n)$ для двух массивов остатков.